Hong Kong Metropolitan University

COMP S209W/8090SEF

Data Structures, Algorithms and Problem Solving
Data Structures, Algorithms

Course Project Report
Group 10

Submission date: __17/2/2026_____

I declare that:
(i) I have read and checked that all parts of the project (including proposal, code programs, and reports), they are contributed by me, here submitted is original except for source material explicitly acknowledged;
(ii) the project, in parts or in whole, has not been submitted for more than one purpose without declaration;
(iii) I am aware of the University's policy and regulations on honesty in academic work and understand the possible consequence when breaching such policy and regulations;
(iv) I confirm that I have declared in the report about the usage of AI and other generative models, including but not limited to ChatGPT, LLaMA, Gemini, Mistral, and Stable Diffusion, and complied with the instructions provided by HKMU; and
(v) I am aware that I should be held responsible and liable to disciplinary actions, irrespective of whether I have signed the declaration and whether I have contributed, directly or indirectly, to the problematic contents.

I confirm that I have read through and understood the project requirements. I understand that failure to comply with the project requirements will result in score deduction.

Name	SID
Ng Chun Yiu	12893986

2. Each student must acknowledge ALL external resources or code from others referenced. We will only mark the parts that are original, not the parts from others.

Smart Restaurant Ordering and Table Management System

GitHub repository link: https://github.com/KaBooMU/COMPS209W-12893986/tree/main/COMP-S209W-CourseProject/

Introduction video link: https://youtu.be/RZsxKb7AmEY

Problem Definition and Real-Life Context

In short, this project aims to solve the problem of manual processing of restaurant orders in a small-scale restaurant, which is not effective. For customers in most segments of the industry, especially if they do not work with a point-of-sale system, it is a fact that employees recall the waiting table, manually write an order, keep track of the number of items that have been altered, and bill their customers manually. It can only be done if customer traffic is low. If two or more tables open for business, the probability of order omissions, pricing mistakes, duplicated information, delay in service, and misunderstanding about the number of sitting tables will be high. I guess the problem isn't so much the facility but the standardization, the consistency, and the quality, the amount of data and accuracy. It's not much of a problem (at least not one that needs to be solved) but instead is that restaurants are implementing integrated point-of-sale systems that automate such things as menu presentation, order registration, table assignment, and billing under one software environment. Business customers often want to know that they are buying, and orders can be made with confidence that each table will fill, and to know when it's empty, an "incorrect information and service error" that the system can create. These systems add both speed and reliability, but they do need databases, interfaces, and commercial infrastructure that are not part of an introductory course project. Hence, we do not attempt to emulate a full commercial system in this project. Instead, Python is used to replicate the same real-life problem in an abstraction model, yet in a technically straightforward manner. For the final submission we are using the original object-oriented Python modules as our business logic layer and add a lightweight Flask-based web interface to interface the user with the system in a browser (as opposed to a very text-based workflow). This design makes sure the course's core learning targets are followed but also adds to usability. The Python modules still write models for customers, menu items, tables, orders, and file storage as independent program units themselves, but the web interface is easier to visualize to the user—it's more like what a real application will look like when implemented. This project is both a manifestation of software design principles and an actual interface improvement.

Description of Core Functions and Summary of Techniques

Finally, the resulting system provides a good model for a small but full restaurant workflow. When the Flask application executes its initial task, it creates a restaurant object, loads an existing menu of food and beverage items, and sets up a small number of tables with varying seating capacity. From that browser interface, the user can place a new order; check whether the table is available, see the table's list; visit the menu, add and delete items; automatically calculate the bill; save the order history on the dashboard; view the saved record for each order. So, the system is not simply a one-feature demo. It simulates the entire buying and selling process of a customer that includes when they have the item in the menu and until they hold the

order data. The project implementation methodology is modular decomposition. This submission separates all this logic into multiple Python modules which represent one unified responsibility of the system instead of placing all this logic in a single script. Customer module provides customer identification, menu module contains common and specialized menu items, table module denotes seating resources available in the restaurant, order module contains transactions and billing, restaurant module controls menu and table operation, and lastly file handler module saves memory and stores order on a persistent basis in a text file. And finally, the Flask app in app.py, which builds those modules, links the browser and Python backend logic. The HTML template is for building a complete form layout for a page, the CSS files for displaying and laying out the form to the web, and JavaScript files for interaction with the website and to fire off Flask routes on JSON requests. Application design uses several of the essential principles of programming techniques. Menu rendering, table traversal, item traversal, and total computation are done in iterations. As a result of this, we have a kind of conditional statement in place where the customer input is used to determine whether a table can contain a party or not and if an order exists before anything else can be done. File input and output are used to save order history after a single run of the app. The browser interface handles requests asynchronously, so the page will modify menu items, status of the table, current order contents and saved history without having the user interact with a plain console. The thing is, it makes the presentation of the project much easier but also holds on to the processing logic of Python modules. A significant design choice here is that billing logic doesn't find its way to the web layer but stays with the order object. The order retains control of some products and takes subtotal, service charge and final total from what is in the current contents of the order. The restaurant object performs a similar function assigning tables; it scans the list of tables in it and returns the one which fits the right size for the party. So the web interface is merely a page rendering and interaction layer; the actual business rules are still enforced according to the original Python classes.

State and Illustrate the Usage of OOP Concepts and Data Structures

The project is implemented on a primary data structure which is the Python list. The list is used in the restaurant menu, in the restaurant table collection, and in the order’s collection of selected items. This is appropriate as these collections are ordered, inerrable, and are subject to dynamic updates at runtime. The menu should allow for repeated traversal and item lookup, the table list should allow for the assignment logic according to capacity and availability, and an order should enable repeated addition and removal of items as the transaction changes. The list thus provides an efficient and conceptually simple structure for these operations. In addition to lists, the system is object-based to model the real-world entities that operate the workflow. A customer is described as an object that represents the customer. As an entity, a table is used as an object representing an item with capacity and occupancy state. The order is modeled based on an object that connects customer, table, chosen selected items, and financial data. The usefulness of this object structure is in the fact that it organizes associated data and behavior together; this makes it easier to understand code, and easier to maintain as well. Flask does not replace those objects so much as, instead, calls and presents their results in the browser. Encapsulation is nicely displayed in the customer and table classes. The customer class contains the customer ID, name, and phone number in private attributes, and they are revealed to the other functions through getter methods. The table class protects the table number, seating capacity, and occupancy at the same time. This is key because state is only altered through an expected interface vs. being tampered with directly. In a layman's sense, encapsulation makes the system more manageable and less open to

invalid states. One way to show inheritance is the menu hierarchy. The base MenuItem class has the general properties attributes of a purchasable item (name, identifier, and price). FoodItem expands this class to include a category, whereas DrinkItem expands this by including a size. This is a good example of inheritance as the two special classes have the same general attribute of a menu item while having attributes of a specific sort. Code reuse is simplified, as common methods and fields are inherited, not rewritten. In the display behavior of menu classes, polymorphism is observed. FoodItem and DrinkItem override the base menu item's display method, providing type-specific output. The original console supported this directly to the terminal; in the web version it matters the same bit though for the same class distinctions are still the factor as the Flask layer converts each item object into structured data for the interface. It illustrates how polymorphism simplifies control logic and keeps responsibility down to the data it describes. The project is also compliant with modular programming and multi-file class organization as the codebase is kept separate into different Python files and the interface layer creates separate HTML, CSS, and JavaScript files in order to display and maintain information more easily.

Evaluation Through Test Cases and Critical Reflection

The correctness of the system was evaluated through a set of representative use cases that reflect normal restaurant operations. In the first test scenario, a new order is created for a party of three through the browser interface. The restaurant object successfully finds the smallest available table that satisfies the seating requirement and marks it as occupied. This confirms that the table assignment logic correctly combines capacity checking with availability checking even when accessed through the Flask route layer.

In the second test scenario, multiple menu items are added to the order through the web interface. The order object then computes the subtotal, the ten per cent service charge, and the final total using the prices stored in the selected menu item objects. The displayed totals match the expected arithmetic values, confirming that billing logic is correctly tied to object state rather than manually entered values. In the third test scenario, one of the selected items is removed. The final bill is recalculated immediately from the current list of items, which confirms that the order object does not depend on a stale cached total. In the fourth test scenario, the order is written to a text file and later displayed in the history panel of the browser interface. The saved content matches the completed transaction, confirming that the persistence mechanism continues to work after the addition of the web layer.

The project succeeds in demonstrating how programming concepts can be used to solve a realistic operational problem in a structured way. It is runnable, modular, and logically coherent. Compared with the earlier console-oriented form, the browser interface improves usability and presentation, which makes the program easier to demonstrate and more suitable for non-technical users. However, the evaluation process also reveals several limitations. Order history is still stored in plain text, which limits scalability, makes structured querying difficult, and offers no protection against concurrent writes or corruption. The current Flask application also assumes a single active order in memory, which is acceptable for demonstration purposes but not sufficient for a real multi-user environment. Input validation has been improved at the route level, but more robust exception handling and session-based state management would still be needed in a production-quality system.

Several future improvements follow naturally from these limitations. The first would be replacing text-file storage with a relational database such as SQLite, which would make saved

order history more reliable, searchable, and scalable. The second would be extending the system to support multiple active orders simultaneously through session management or a database-backed order model. The third would be enhancing input validation and error handling even further so that the application can reject malformed requests more gracefully. Finally, the system could be extended to support richer restaurant functions such as order status transitions, staff roles, or management reports. These improvements are beyond the current project scope, but they represent realistic next steps in the evolution of the prototype.

Appendix A. Tables for Task 1

Table A1 summarises the functional test cases used to evaluate the correctness of the implemented system.

Table A1. Functional test cases

Test Case	Scenario	Input	Expected Outcome	Actual Outcome	Status
TC001	Create a new order through web UI	3 diners	A table with capacity ≥ 3 is assigned and marked occupied	Table 2 is assigned correctly and table status updates on the page	Passed
TC002	Add two items and compute bill	Burger + Coffee	Subtotal, service charge and final total are computed from item prices	All three values are computed correctly in the order panel	Passed
TC003	Remove one item from order	Remove Coffee	Final total is recalculated from remaining items	Updated total is shown correctly after removal	Passed
TC004	Save and read order history	Current order	A text record is appended and can be read later	Record is written to orders.txt and displayed in history panel	Passed

References

Python Software Foundation. (n.d.). Python documentation. https://docs.python.org/3/
Pallets. (n.d.). Flask documentation. https://flask.palletsprojects.com/
Hong Kong Metropolitan University. (2026). COMP S209W Data Structures, Algorithms and Problem Solving lecture notes.